

Redis Cache Setup and Usage Guide

Complete guide to setting up and using Redis as a caching layer.

What is Redis?

Redis = Remote Dictionary Server = In-memory data structure store

Key Features:

- In-memory storage (very fast)
- Key-value store
- Supports multiple data types (strings, lists, sets, hashes)
- Persistence options (snapshot, append-only file)
- Pub/Sub messaging
- Atomic operations
- TTL (Time To Live) for automatic expiration

Use Cases:

- Caching (most common)
- Session storage
- Real-time analytics
- Message queues
- Rate limiting
- Leaderboards

Installation

Windows

Option 1: Using WSL (Recommended)

```
# Install WSL
wsl --install

# In WSL, install Redis
sudo apt update
sudo apt install redis-server

# Start Redis
sudo service redis-server start

# Test
redis-cli ping
# Should return: PONG
```

Option 2: Redis Windows Port (Not recommended for production)

```
# Download from: https://github.com/tporadowski/redis/releases
# Extract and run redis-server.exe

# Or using Chocolatey
choco install redis-64
```

Linux (Ubuntu/Debian)

```
# Update packages
sudo apt update

# Install Redis
sudo apt install redis-server

# Start Redis
sudo systemctl start redis-server

# Enable on boot
sudo systemctl enable redis-server

# Check status
sudo systemctl status redis-server

# Test
redis-cli ping
# Should return: PONG
```

macOS

```
# Using Homebrew
brew install redis

# Start Redis
brew services start redis

# Or run manually
redis-server

# Test
redis-cli ping
```

Docker (All Platforms)

```
# Pull Redis image
docker pull redis:latest

# Run Redis container
docker run -d \
  --name redis \
  -p 6379:6379 \
  redis:latest

# Test
docker exec -it redis redis-cli ping
# Should return: PONG

# With persistence
docker run -d \
  --name redis \
  -p 6379:6379 \
  -v redis-data:/data \
  redis:latest redis-server --appendonly yes
```

Docker Compose

```
# docker-compose.yml
version: '3.8'

services:
  redis:
    image: redis:latest
    container_name: redis
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    command: redis-server --appendonly yes
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 3

volumes:
  redis-data:
```

```
# Start
docker-compose up -d
```

```
# Check
docker-compose ps
```

Configuration

Basic Configuration (redis.conf)

```
# Find config file
# Linux: /etc/redis/redis.conf
# macOS: /usr/local/etc/redis.conf
# Docker: Copy from container

# Important settings
bind 127.0.0.1           # Listen on localhost only (secure)
port 6379               # Default port
daemonize yes           # Run as daemon (background)
maxmemory 256mb         # Max memory limit
maxmemory-policy allkeys-lru # Eviction policy when memory full

# Persistence
appendonly yes          # Enable AOF (append-only file)
appendfsync everysec    # Sync every second (balance performance/durability)

# Password (security)
requirepass your_strong_password

# Restart Redis after config changes
sudo systemctl restart redis-server
```

Environment-Specific Configs

Development:

```
# redis-dev.conf
bind 0.0.0.0
protected-mode no
maxmemory 128mb
appendonly no # Faster, no persistence needed
```

Production:

```
# redis-prod.conf
bind 127.0.0.1
protected-mode yes
requirepass strong_password_here
maxmemory 2gb
```

```
maxmemory-policy allkeys-lru
appendonly yes
appendfsync everysec
```

Python Integration

Install Redis Client

```
pip install redis
```

Basic Usage

```
import redis

# Connect to Redis
r = redis.Redis(
    host='localhost',
    port=6379,
    db=0, # Database number (0-15)
    decode_responses=True # Return strings, not bytes
)

# Test connection
r.ping() # Returns True

# Set key-value
r.set('name', 'Alice')

# Get value
name = r.get('name') # Returns 'Alice'

# Set with expiration (TTL in seconds)
r.setex('session_token', 3600, 'abc123') # Expires in 1 hour

# Check if key exists
r.exists('name') # Returns 1 (true)

# Delete key
r.delete('name')

# Set multiple keys
r.mset({'key1': 'value1', 'key2': 'value2'})

# Get multiple keys
values = r.mget('key1', 'key2') # Returns ['value1', 'value2']
```

Caching Decorator

```
import redis
import json
import functools
from typing import Any, Callable

# Redis connection
redis_client = redis.Redis(host='localhost', port=6379, decode_responses=True)

def cache(ttl: int = 300):
    """
    Caching decorator

    Args:
        ttl: Time to live in seconds (default: 5 minutes)
    """
    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        def wrapper(*args, **kwargs) -> Any:
            # Generate cache key from function name and arguments
            cache_key = f"{func.__name__}:{str(args)}:{str(kwargs)}"

            # Try to get from cache
            cached_value = redis_client.get(cache_key)
            if cached_value:
                print(f"Cache HIT: {cache_key}")
                return json.loads(cached_value)

            # Cache miss - execute function
            print(f"Cache MISS: {cache_key}")
            result = func(*args, **kwargs)

            # Store in cache
            redis_client.setex(
                cache_key,
                ttl,
                json.dumps(result)
            )

            return result
        return wrapper
    return decorator

# Usage
@cache(ttl=600) # Cache for 10 minutes
def get_user(user_id: int):
    """Expensive database query"""
    print(f"Fetching user {user_id} from database...")
    # Simulate database query
    import time
    time.sleep(2)
```

```

    return {
        "id": user_id,
        "name": "Alice",
        "email": "alice@example.com"
    }

# First call - cache miss, hits database
user = get_user(123) # Takes 2 seconds
# Output: Cache MISS: get_user:(123,):{}
#         Fetching user 123 from database...

# Second call - cache hit, returns immediately
user = get_user(123) # Returns instantly
# Output: Cache HIT: get_user:(123,):{}

```

FastAPI with Redis Cache

```

from fastapi import FastAPI, Depends
import redis
import json
from typing import Optional

app = FastAPI()

# Redis connection pool
redis_pool = redis.ConnectionPool(
    host='localhost',
    port=6379,
    db=0,
    decode_responses=True,
    max_connections=10
)

def get_redis():
    return redis.Redis(connection_pool=redis_pool)

@app.get("/users/{user_id}")
async def get_user(user_id: int, r: redis.Redis = Depends(get_redis)):
    cache_key = f"user:{user_id}"

    # Check cache
    cached = r.get(cache_key)
    if cached:
        return {"source": "cache", "data": json.loads(cached)}

    # Cache miss - fetch from database
    user = fetch_from_database(user_id) # Your DB logic

    # Store in cache (5 minutes TTL)
    r.setex(cache_key, 300, json.dumps(user))

```

```

        return {"source": "database", "data": user}

@app.delete("/users/{user_id}")
async def delete_user(user_id: int, r: redis.Redis = Depends(get_redis)):
    # Delete from database
    delete_from_database(user_id)

    # Invalidate cache
    r.delete(f"user:{user_id}")

    return {"status": "deleted"}

```

Cache-Aside Pattern (Most Common)

```

class UserCache:
    def __init__(self, redis_client):
        self.redis = redis_client
        self.ttl = 300 # 5 minutes

    def get_user(self, user_id: int):
        """Get user with cache-aside pattern"""
        cache_key = f"user:{user_id}"

        # 1. Try cache first
        cached = self.redis.get(cache_key)
        if cached:
            return json.loads(cached)

        # 2. Cache miss - fetch from database
        user = self._fetch_from_db(user_id)

        # 3. Store in cache for next time
        if user:
            self.redis.setex(cache_key, self.ttl, json.dumps(user))

        return user

    def update_user(self, user_id: int, data: dict):
        """Update user and invalidate cache"""
        # 1. Update database
        self._update_db(user_id, data)

        # 2. Invalidate cache
        cache_key = f"user:{user_id}"
        self.redis.delete(cache_key)

        # Or update cache immediately (write-through)
        # self.redis.setex(cache_key, self.ttl, json.dumps(data))

    def _fetch_from_db(self, user_id: int):
        # Your database logic

```

```
pass

def _update_db(self, user_id: int, data: dict):
    # Your database logic
    pass
```

Advanced Caching Patterns

1. Cache with Automatic Refresh

```
import time
from threading import Thread

class RefreshingCache:
    def __init__(self, redis_client):
        self.redis = redis_client

    def get_with_refresh(self, key: str, fetch_func, ttl: int = 300):
        """Get from cache, refresh in background if stale"""
        cached = self.redis.get(key)

        # Check if cache exists
        if cached:
            # Check TTL remaining
            remaining = self.redis.ttl(key)

            # If TTL < 25% of original, refresh in background
            if remaining < ttl * 0.25:
                Thread(target=self._refresh_cache, args=(key, fetch_func,
ttl)).start()

            return json.loads(cached)

        # Cache miss
        data = fetch_func()
        self.redis.setex(key, ttl, json.dumps(data))
        return data

    def _refresh_cache(self, key, fetch_func, ttl):
        """Background refresh"""
        try:
            data = fetch_func()
            self.redis.setex(key, ttl, json.dumps(data))
        except Exception as e:
            print(f"Cache refresh failed: {e}")
```

2. Cache with Circuit Breaker

```

from datetime import datetime, timedelta

class CacheWithCircuitBreaker:
    def __init__(self, redis_client):
        self.redis = redis_client
        self.failure_count = 0
        self.failure_threshold = 3
        self.recovery_timeout = 60
        self.last_failure = None
        self.state = "closed" # closed, open, half-open

    def get(self, key: str):
        # If circuit open and timeout not passed
        if self.state == "open":
            if datetime.now() - self.last_failure <
timedelta(seconds=self.recovery_timeout):
                return None # Skip cache, go to database
            else:
                self.state = "half-open" # Try again

        try:
            value = self.redis.get(key)

            # Success - reset circuit
            if self.state == "half-open":
                self.state = "closed"
                self.failure_count = 0

            return value

        except redis.RedisError:
            self.failure_count += 1

            if self.failure_count >= self.failure_threshold:
                self.state = "open"
                self.last_failure = datetime.now()
                print("Circuit breaker OPEN - Redis unavailable")

            return None # Fall back to database

```

3. Multi-Level Cache (L1: Memory, L2: Redis)

```

from functools import lru_cache
import redis

class MultiLevelCache:
    def __init__(self, redis_client):
        self.redis = redis_client
        self.l1_cache = {} # In-memory cache
        self.l1_max_size = 100

```

```

        self.l1_ttl = 60 # 1 minute
        self.l2_ttl = 600 # 10 minutes

    def get(self, key: str, fetch_func):
        # L1 Cache (memory)
        if key in self.l1_cache:
            entry = self.l1_cache[key]
            if time.time() - entry['timestamp'] < self.l1_ttl:
                print(f"L1 HIT: {key}")
                return entry['value']
            else:
                del self.l1_cache[key]

        # L2 Cache (Redis)
        cached = self.redis.get(key)
        if cached:
            print(f"L2 HIT: {key}")
            value = json.loads(cached)

            # Promote to L1
            self._set_l1(key, value)
            return value

        # Cache miss - fetch from source
        print(f"MISS: {key}")
        value = fetch_func()

        # Store in both caches
        self._set_l1(key, value)
        self.redis.setex(key, self.l2_ttl, json.dumps(value))

        return value

    def _set_l1(self, key: str, value):
        # Evict oldest if full
        if len(self.l1_cache) >= self.l1_max_size:
            oldest = min(self.l1_cache.items(), key=lambda x: x[1]['timestamp'])
            del self.l1_cache[oldest[0]]

        self.l1_cache[key] = {
            'value': value,
            'timestamp': time.time()
        }

```

Redis Data Types

1. Strings (Most Common)

```

# Simple key-value
r.set('key', 'value')

```

```
r.get('key')

# With expiration
r.setex('key', 3600, 'value') # 1 hour

# Increment counter
r.set('page_views', 0)
r.incr('page_views') # 1
r.incr('page_views') # 2
r.incrby('page_views', 10) # 12
```

2. Hashes (Objects)

```
# Store user object
r.hset('user:123', mapping={
    'name': 'Alice',
    'email': 'alice@example.com',
    'age': 30
})

# Get single field
name = r.hget('user:123', 'name') # 'Alice'

# Get all fields
user = r.hgetall('user:123')
# {'name': 'Alice', 'email': 'alice@example.com', 'age': '30'}

# Update field
r.hset('user:123', 'age', 31)

# Increment field
r.hincrby('user:123', 'login_count', 1)
```

3. Lists (Queues)

```
# Add to list
r.lpush('tasks', 'task1') # Left push
r.rpush('tasks', 'task2') # Right push

# Get from list
task = r.lpop('tasks') # Pop from left (FIFO queue)
task = r.rpop('tasks') # Pop from right (LIFO stack)

# Get range
tasks = r.lrange('tasks', 0, -1) # Get all

# List length
count = r.llen('tasks')
```

4. Sets (Unique Values)

```
# Add to set
r.sadd('tags', 'python', 'redis', 'cache')

# Check membership
r.sismember('tags', 'python') # True

# Get all members
tags = r.smembers('tags') # {'python', 'redis', 'cache'}

# Set operations
r.sadd('set1', 'a', 'b', 'c')
r.sadd('set2', 'b', 'c', 'd')

r.union('set1', 'set2') # Union: {'a', 'b', 'c', 'd'}
r.sinter('set1', 'set2') # Intersection: {'b', 'c'}
r.sdiff('set1', 'set2') # Difference: {'a'}
```

5. Sorted Sets (Leaderboards)

```
# Add with score
r.zadd('leaderboard', {'alice': 100, 'bob': 85, 'charlie': 92})

# Get top 3
top = r.zrevrange('leaderboard', 0, 2, withscores=True)
# [('alice', 100.0), ('charlie', 92.0), ('bob', 85.0)]

# Get rank
rank = r.zrevrank('leaderboard', 'alice') # 0 (first place)

# Increment score
r.zincrby('leaderboard', 10, 'bob') # bob now has 95
```

Cache Strategies

1. Cache-Aside (Lazy Loading)

Most common pattern

```
def get_user(user_id):
    # 1. Check cache
    cached = redis.get(f'user:{user_id}')
    if cached:
        return json.loads(cached)

    # 2. Cache miss - query database
```

```
user = db.query("SELECT * FROM users WHERE id = ?", user_id)

# 3. Store in cache
redis.setex(f'user:{user_id}', 3600, json.dumps(user))

return user
```

Pros: Only caches what's needed **Cons:** Cache misses are slower (need DB query)

2. Write-Through

Write to cache AND database simultaneously

```
def update_user(user_id, data):
    # 1. Update database
    db.execute("UPDATE users SET ... WHERE id = ?", user_id)

    # 2. Update cache immediately
    redis.setex(f'user:{user_id}', 3600, json.dumps(data))
```

Pros: Cache always consistent **Cons:** Write latency (2 operations)

3. Write-Behind (Write-Back)

Write to cache first, sync to database later

```
def update_user(user_id, data):
    # 1. Update cache immediately
    redis.setex(f'user:{user_id}', 3600, json.dumps(data))

    # 2. Queue database update (async)
    task_queue.enqueue(sync_to_database, user_id, data)
```

Pros: Fast writes **Cons:** Risk of data loss if cache fails before sync

4. Read-Through

Cache handles both cache and database reads

```
class ReadThroughCache:
    def get(self, key, loader_func):
        cached = redis.get(key)
        if cached:
            return json.loads(cached)

        # Loader function fetches from DB
        data = loader_func()
```

```
        redis.setex(key, 3600, json.dumps(data))
    return data

# Usage
cache = ReadThroughCache()
user = cache.get('user:123', lambda: fetch_user_from_db(123))
```

5. Refresh-Ahead

Refresh cache before expiration

```
def get_with_refresh_ahead(key, fetch_func, ttl=300):
    cached = redis.get(key)
    remaining_ttl = redis.ttl(key)

    # Refresh if TTL < 25% remaining
    if cached and remaining_ttl < ttl * 0.25:
        # Refresh in background
        threading.Thread(target=lambda: redis.setex(key, ttl,
            fetch_func())).start()

    if cached:
        return json.loads(cached)

    data = fetch_func()
    redis.setex(key, ttl, json.dumps(data))
    return data
```

Cache Invalidation

Two hardest things in computer science:

1. Cache invalidation
2. Naming things
3. Off-by-one errors

Pattern 1: TTL-Based (Time-To-Live)

```
# Cache expires automatically after TTL
redis.setex('key', 3600, value) # Expires in 1 hour
```

Pros: Simple, automatic **Cons:** Stale data until expiration

Pattern 2: Event-Based Invalidation

```
# Invalidate on update
def update_user(user_id, data):
    db.update_user(user_id, data)
    redis.delete(f'user:{user_id}') # Invalidate cache

# Invalidate related caches
def update_order(order_id, data):
    db.update_order(order_id, data)

    # Invalidate order cache
    redis.delete(f'order:{order_id}')

    # Invalidate user's orders list
    user_id = data['user_id']
    redis.delete(f'user:{user_id}:orders')
```

Pattern 3: Cache Tags

```
# Tag-based invalidation
def cache_with_tags(key, value, tags, ttl=3600):
    # Store value
    redis.setex(key, ttl, json.dumps(value))

    # Store tags
    for tag in tags:
        redis.sadd(f'tag:{tag}', key)

def invalidate_tag(tag):
    # Get all keys with this tag
    keys = redis.smembers(f'tag:{tag}')

    # Delete all
    if keys:
        redis.delete(*keys)

    # Delete tag set
    redis.delete(f'tag:{tag}')

# Usage
cache_with_tags('user:123', user_data, tags=['user', 'user:123'])
cache_with_tags('user:123:orders', orders, tags=['user', 'user:123', 'orders'])

# Invalidate all user caches
invalidate_tag('user:123')
```

Pattern 4: Cache Versioning

```
# Version-based cache keys
def get_cache_key(base_key, version):
    return f'{base_key}:v{version}'

# Store current version
redis.set('user:schema_version', 1)

# Get with version
version = redis.get('user:schema_version')
cache_key = get_cache_key('user:123', version)
user = redis.get(cache_key)

# When schema changes, increment version
# All old caches automatically "invalidated" (different key)
redis.incr('user:schema_version') # Now version 2
```

Performance Optimization

1. Connection Pooling

```
# BAD: Creating new connection each time
def get_user(user_id):
    r = redis.Redis(host='localhost', port=6379)
    return r.get(f'user:{user_id}')

# GOOD: Use connection pool
pool = redis.ConnectionPool(
    host='localhost',
    port=6379,
    max_connections=10
)

def get_user(user_id):
    r = redis.Redis(connection_pool=pool)
    return r.get(f'user:{user_id}')
```

2. Pipeline (Batch Operations)

```
# BAD: Multiple round trips
for i in range(100):
    redis.set(f'key{i}', f'value{i}') # 100 network calls

# GOOD: Single round trip
pipe = redis.pipeline()
for i in range(100):
    pipe.set(f'key{i}', f'value{i}')
pipe.execute() # 1 network call
```

3. Lua Scripts (Atomic Operations)

```
# Atomic increment with limit
lua_script = """
local current = redis.call('GET', KEYS[1])
if current and tonumber(current) >= tonumber(ARGV[1]) then
    return 0
else
    return redis.call('INCR', KEYS[1])
end
"""

# Increment with max limit of 100
increment_with_limit = redis.register_script(lua_script)
result = increment_with_limit(keys=['counter'], args=[100])
```

4. Compression (Large Values)

```
import gzip
import json

def compress_cache_value(data):
    json_str = json.dumps(data)
    compressed = gzip.compress(json_str.encode())
    return compressed

def decompress_cache_value(compressed):
    decompressed = gzip.decompress(compressed)
    return json.loads(decompressed.decode())

# Store compressed
data = {'large': 'data' * 1000}
compressed = compress_cache_value(data)
redis.setex('key', 3600, compressed)

# Retrieve and decompress
compressed = redis.get('key')
data = decompress_cache_value(compressed)
```

Monitoring and Management

Redis CLI Commands

```
# Connect to Redis
redis-cli
```

```
# Test connection
PING

# Get info
INFO

# Memory stats
INFO memory

# Get all keys (DON'T USE IN PRODUCTION)
KEYS *

# Better: Scan with pattern
SCAN 0 MATCH user:* COUNT 100

# Get key type
TYPE user:123

# Get TTL
TTL user:123

# Monitor commands in real-time
MONITOR

# Get slow queries
SLOWLOG GET 10

# Memory usage of key
MEMORY USAGE user:123

# Flush all data (DANGEROUS)
FLUSHALL

# Flush current database only
FLUSHDB
```

Python Monitoring

```
import redis

r = redis.Redis()

# Server info
info = r.info()
print(f"Redis version: {info['redis_version']}")
print(f"Used memory: {info['used_memory_human']}")
print(f"Connected clients: {info['connected_clients']}")
print(f"Total commands: {info['total_commands_processed']}")

# Key statistics
```

```
def get_key_stats():
    info = r.info('keyspace')
    return info

# Memory usage
def get_memory_usage():
    info = r.info('memory')
    return {
        'used_memory': info['used_memory_human'],
        'used_memory_peak': info['used_memory_peak_human'],
        'mem_fragmentation_ratio': info['mem_fragmentation_ratio']
    }

# Slow log
def get_slow_queries(count=10):
    return r.slowlog_get(count)
```

Prometheus Metrics

```
from prometheus_client import Counter, Histogram, Gauge
import redis
import time

# Metrics
redis_cache_hits = Counter('redis_cache_hits_total', 'Cache hits')
redis_cache_misses = Counter('redis_cache_misses_total', 'Cache misses')
redis_operation_duration = Histogram('redis_operation_duration_seconds',
    'Operation duration')
redis_connection_errors = Counter('redis_connection_errors_total', 'Connection
errors')

def get_from_cache(key):
    start = time.time()

    try:
        value = redis_client.get(key)

        # Record duration
        duration = time.time() - start
        redis_operation_duration.observe(duration)

        if value:
            redis_cache_hits.inc()
        else:
            redis_cache_misses.inc()

        return value

    except redis.RedisError:
        redis_connection_errors.inc()
        return None
```

```
# Hit rate calculation
def get_cache_hit_rate():
    hits = redis_cache_hits._value.get()
    misses = redis_cache_misses._value.get()
    total = hits + misses

    if total == 0:
        return 0

    return (hits / total) * 100
```

Production Best Practices

1. Security

```
# Use password
r = redis.Redis(
    host='localhost',
    port=6379,
    password='your_strong_password',
    ssl=True # Use SSL/TLS in production
)

# Limit network access
# redis.conf:
# bind 127.0.0.1 # Only localhost
# requirepass your_password
```

2. High Availability (Redis Sentinel)

```
from redis.sentinel import Sentinel

# Sentinel setup
sentinel = Sentinel([
    ('sentinel1.example.com', 26379),
    ('sentinel2.example.com', 26379),
    ('sentinel3.example.com', 26379)
], socket_timeout=0.1)

# Get master
master = sentinel.master_for('mymaster', socket_timeout=0.1)
master.set('key', 'value')

# Get slave (read-only)
slave = sentinel.slave_for('mymaster', socket_timeout=0.1)
value = slave.get('key')
```

3. Redis Cluster (Sharding)

```
from rediscluster import RedisCluster

# Cluster setup
startup_nodes = [
    {"host": "node1.example.com", "port": 7000},
    {"host": "node2.example.com", "port": 7001},
    {"host": "node3.example.com", "port": 7002}
]

rc = RedisCluster(startup_nodes=startup_nodes, decode_responses=True)

# Use like normal Redis
rc.set('key', 'value')
value = rc.get('key')
```

4. Backup and Persistence

```
# Enable AOF (Append-Only File)
appendonly yes
appendfsync everysec

# Or use RDB snapshots
save 900 1      # Save if 1 key changed in 900 seconds
save 300 10     # Save if 10 keys changed in 300 seconds
save 60 10000   # Save if 10000 keys changed in 60 seconds

# Manual backup
BGSAVE

# Restore
# Copy dump.rdb to Redis data directory
# Restart Redis
```

5. Memory Management

```
# Set memory limit and eviction policy
# redis.conf:
maxmemory 2gb
maxmemory-policy allkeys-lru

# Eviction policies:
# - noeviction: Return error when memory limit reached
# - allkeys-lru: Evict least recently used keys
# - volatile-lru: Evict LRU keys with TTL set
# - allkeys-random: Evict random keys
```

```
# - volatile-random: Evict random keys with TTL  
# - volatile-ttl: Evict keys with shortest TTL
```

Troubleshooting

Issue 1: Slow Performance

```
# Check slow queries  
redis-cli SLOWLOG GET 10  
  
# Check memory  
redis-cli INFO memory  
  
# Check connected clients  
redis-cli CLIENT LIST  
  
# Monitor commands  
redis-cli MONITOR
```

Solutions:

- Use pipelining for bulk operations
- Enable connection pooling
- Use appropriate data structures
- Add indexes if using RedisSearch

Issue 2: High Memory Usage

```
# Find largest keys  
redis-cli --bigkeys  
  
# Memory usage of specific key  
redis-cli MEMORY USAGE user:123
```

Solutions:

- Set TTL on all keys
- Use compression for large values
- Configure maxmemory and eviction policy
- Consider Redis Cluster for sharding

Issue 3: Connection Errors

```
try:  
    r.ping()
```

```
except redis.ConnectionError:
    print("Redis connection failed")
    # Fallback to database
```

Solutions:

- Check Redis is running: `sudo systemctl status redis`
- Check firewall rules
- Verify host and port
- Use circuit breaker pattern

Issue 4: Cache Stampede

Problem: Many requests hit expired cache simultaneously, all query database

```
# Solution: Lock pattern
import uuid

def get_with_lock(key, fetch_func, ttl=300):
    cached = redis.get(key)
    if cached:
        return json.loads(cached)

    # Try to acquire lock
    lock_key = f'{key}:lock'
    lock_value = str(uuid.uuid4())

    # SET with NX (only if not exists) and EX (expiration)
    acquired = redis.set(lock_key, lock_value, nx=True, ex=10)

    if acquired:
        try:
            # We have the lock - fetch data
            data = fetch_func()
            redis.setex(key, ttl, json.dumps(data))
            return data
        finally:
            # Release lock
            redis.delete(lock_key)
    else:
        # Someone else is fetching - wait and retry
        time.sleep(0.1)
        return get_with_lock(key, fetch_func, ttl)
```

Complete Example: FastAPI + Redis Cache

```
from fastapi import FastAPI, Depends, HTTPException
import redis
```

```
import json
from typing import Optional
from functools import wraps
import time

app = FastAPI()

# Redis connection pool
redis_pool = redis.ConnectionPool(
    host='localhost',
    port=6379,
    password='your_password',
    decode_responses=True,
    max_connections=10
)

def get_redis():
    """Dependency to get Redis client"""
    return redis.Redis(connection_pool=redis_pool)

def cached(ttl: int = 300):
    """Cache decorator for FastAPI endpoints"""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            # Get Redis client from dependencies
            r = kwargs.get('r') or kwargs.get('redis')

            # Generate cache key
            cache_key = f"{func.__name__}:{str(args)}:{str(kwargs)}"

            # Try cache
            if r:
                cached_value = r.get(cache_key)
                if cached_value:
                    return json.loads(cached_value)

            # Execute function
            result = await func(*args, **kwargs)

            # Store in cache
            if r:
                r.setex(cache_key, ttl, json.dumps(result))

            return result
        return wrapper
    return decorator

# Simulated database
fake_db = {
    1: {"id": 1, "name": "Alice", "email": "alice@example.com"},
    2: {"id": 2, "name": "Bob", "email": "bob@example.com"}
}
```

```
@app.get("/users/{user_id}")
@cached(ttl=300)
async def get_user(user_id: int, r: redis.Redis = Depends(get_redis)):
    """Get user with caching"""
    # Simulate slow database query
    time.sleep(1)

    user = fake_db.get(user_id)
    if not user:
        raise HTTPException(404, "User not found")

    return user

@app.put("/users/{user_id}")
async def update_user(
    user_id: int,
    name: str,
    email: str,
    r: redis.Redis = Depends(get_redis)
):
    """Update user and invalidate cache"""
    # Update database
    fake_db[user_id] = {
        "id": user_id,
        "name": name,
        "email": email
    }

    # Invalidate cache
    cache_key = f"get_user:({user_id},):*"
    for key in r.scan_iter(match=cache_key):
        r.delete(key)

    return {"status": "updated"}

@app.get("/users")
async def list_users(r: redis.Redis = Depends(get_redis)):
    """List all users with caching"""
    cache_key = "users:list"

    # Try cache
    cached = r.get(cache_key)
    if cached:
        return {"source": "cache", "users": json.loads(cached)}

    # Cache miss
    users = list(fake_db.values())

    # Store in cache (5 minutes)
    r.setex(cache_key, 300, json.dumps(users))

    return {"source": "database", "users": users}

@app.get("/stats")
```

```

async def get_stats(r: redis.Redis = Depends(get_redis)):
    """Get Redis statistics"""
    info = r.info()

    return {
        "redis_version": info['redis_version'],
        "used_memory": info['used_memory_human'],
        "connected_clients": info['connected_clients'],
        "total_commands": info['total_commands_processed'],
        "keyspace": r.dbsize()
    }

@app.delete("/cache")
async def clear_cache(r: redis.Redis = Depends(get_redis)):
    """Clear all cache"""
    r.flushdb()
    return {"status": "cache cleared"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Test the API:

```

# First request (cache miss, slow)
curl http://localhost:8000/users/1
# Takes 1 second

# Second request (cache hit, fast)
curl http://localhost:8000/users/1
# Returns instantly

# Update user
curl -X PUT "http://localhost:8000/users/1?name=Alice2&email=new@example.com"

# Get again (cache invalidated, slow)
curl http://localhost:8000/users/1
# Takes 1 second (cache was invalidated)

# Get stats
curl http://localhost:8000/stats

# Clear cache
curl -X DELETE http://localhost:8000/cache

```

Interview Questions & Answers

Q: What is Redis and when would you use it?

A: "Redis is an in-memory key-value store used primarily for caching. Use it when:

- Need fast data access (< 1ms)
- Frequently accessed data (user sessions, API responses)
- Temporary data (cache, rate limiting)
- Real-time features (leaderboards, pub/sub)

Example: Cache database queries - first request hits DB (slow), subsequent requests hit Redis cache (fast), significantly reducing database load and response time."

Q: How do you handle cache invalidation?

A: "Three strategies:

1. TTL-based: Set expiration time, cache auto-expires
2. Event-based: Invalidate on data update
3. Cache tags: Group related caches, invalidate by tag

Example: User profile cached for 5 minutes (TTL). When user updates profile, explicitly delete cache key. For complex cases, use tags to invalidate user's profile, orders, and preferences together."

Q: What happens if Redis goes down?

A: "Implement fallback strategy:

1. Circuit breaker: Skip cache if Redis unavailable
2. Try cache, catch exception, fallback to database
3. Use Redis Sentinel for automatic failover

Code:

```
try:
    value = redis.get(key)
    if value: return value
except redis.RedisError:
    pass # Fall through to database

return database.query(key)
```

Critical: Application should work without cache, just slower."

Q: Redis vs Memcached?

A: "Redis advantages:

- More data types (lists, sets, sorted sets)
- Persistence options
- Pub/sub messaging
- Atomic operations
- Built-in replication

Memcached advantages:

- Simpler (just cache)
- Slightly faster for simple key-value
- Multi-threaded

Choose Redis for production - more features, same performance."

Summary

Setup:

- Install: Docker (recommended) or native
- Configure: maxmemory, eviction policy, password
- Connect: Connection pooling

Usage:

- Cache-aside pattern (most common)
- TTL for automatic expiration
- Invalidate on updates

Optimization:

- Connection pooling
- Pipelining for bulk ops
- Compression for large values

Production:

- Monitor with Prometheus
- Use Redis Sentinel/Cluster
- Set memory limits
- Enable persistence

Key Takeaway: Redis is fast, simple, and powerful. Start with basic cache-aside pattern, add complexity only when needed.